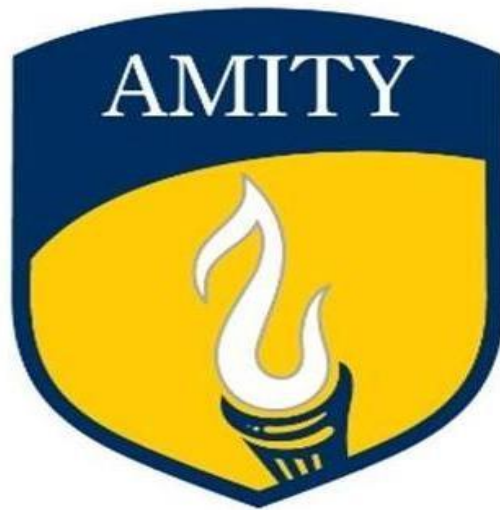


AMITY UNIVERSITY HARYANA
AMITY SCHOOL OF ENGINEERING AND
TECHNOLOGY



Data Warehousing And Multidimensional
Modelling Lab(AIEU1542)

Submitted By :

Abhishek Dixit

M.Tech(AI)

A501144825003

Submitted To :

Dr. Sarita Gulia

Experiment No	Title	Date	Page no
1	To understand and implement basic SQL commands in database.		
2	Introduction to PL/SQL and Its Features.		
3	To create a PL/SQL procedure that prints a custom message (“Hello World”) on the screen.		
4	To create a PL/SQL Program to Add Two Numbers.		
5	To create a PL/SQL Program to Check Number is Odd or Even.		
6	To find the greatest of two numbers using PL/SQL.		
7	To implement loops in PL/SQL.		
8	To implement operators in PL/SQL.		
9	To implement Switch Case Statement in PL/SQL.		
10	To implement exception handling in PL/SQL.		
11	To implement various string functions in PL/SQL.		

12	To implement functions and procedure in PL/SQL.		
13	To implement triggers in PL/SQL.		
14	To showcase a PL/SQL-based project.		

Practical No.: 01

AIM: To understand and implement basic SQL commands in database (creating, inserting, retrieving, updating, and deleting data in a database etc.)

SQL (Structured Query Language) is a standard language used to interact with Relational Database Management Systems (RDBMS) like MySQL, Oracle, and SQL Server. It helps in creating, managing, and manipulating

Command	Purpose	Example
CREATE	To create a new table or database	CREATE TABLE Students (ID INT, Name VARCHAR(50)); INSERT INTO Students VALUES (1,
INSERT	To insert data into a table	'Kunal');
SELECT	To retrieve data from a table	SELECT * FROM Students; UPDATE Students SET Name = 'Kunal'
UPDATE	To modify existing data	WHERE ID = 1;
DELETE	To delete data from a table	DELETE FROM Students WHERE ID = 1;
DROP	To delete a table or database	DROP TABLE Students;
ALTER	To modify the structure of a table	ALTER TABLE Students ADD Age INT;
WHERE	To filter records based on condition	SELECT * FROM Students WHERE ID = 1; SELECT * FROM Students ORDER BY
ORDER BY	To sort the result	Name ASC;

	To group rows with the same	SELECT COUNT(*), Age FROM
GROUP BY	Students	
	values	GROUP BY Age;
		SELECT COUNT(*), Age FROM Students
	To filter groups	
HAVING		GROUP BY Age HAVING COUNT(*) > 1;

CREATE (Creating a table)

```
1 CREATE TABLE Students (  
2     id INT PRIMARY KEY,  
3     name VARCHAR(50),  
4     age INT,  
5     course VARCHAR(50)  
6 );
```

INSERT (Adding Value)

```
7 INSERT INTO Students (id, name, age, course)
8 VALUES (1, 'Rahul', 22, 'AI');
9
10 INSERT INTO Students (id, name, age, course)
11 VALUES (2, 'Priya', 21, 'Data Science');
12 select* from Students
```

RETRIEVE (Show table)

```
13 -- Retrieve data
14 SELECT * FROM Students;
15
```

OUTPUT

```
1|Rahul|22|AI
2|Priya|21|Data Science
```

UPDATE (Modifying Data)

```
16 -- Update data
17 UPDATE Students
18 SET age = 23
19 WHERE id = 1;
20 select * From Students
```

OUTPUT

```
1|Rahul|23|AI
2|Priya|21|Data Science
```

DELETE (Removing Data)

```
-- Delete data  
DELETE FROM Students  
WHERE id = 2;  
  
SELECT * FROM Students;
```

OUTPUT

```
1|Rahul|23|AI
```

```
[Execution complete with exit code 0]
```

Practical No.: 02

AIM: Introduction to PL/SQL and Its Features.

PL/SQL (Procedural Language/SQL) is an extension of SQL used in Oracle Database. It allows you to combine SQL queries with programming features like variables, loops, and conditions.

It is mainly used to process data inside the database, write stored procedures, functions, and triggers, build secure and efficient database applications. In simple terms PL/SQL= SQL+ Programming logic.

Features of PL/SQL

Combination of SQL and Programming

- Allows use of SQL (SELECT, INSERT, UPDATE, DELETE) with logic
- Example: conditions, loops

Block Structure

- Code is organized into blocks (DECLARE, BEGIN, END)
- Improves readability and maintenance

Variables and Data Types

- You can store data temporarily using variables

Control Structures

- Supports decision-making and loops
- IF-ELSE, WHILE, FOR loops

Error Handling (Exception Handling)

- Handles runtime errors using EXCEPTION block

Triggers

- Automatically executed when events occur (INSERT, UPDATE, DELETE)

Stored Procedures and Functions

- Reusable blocks of code stored in database

Practical No.: 03

AIM: To create a PL/SQL procedure that prints a custom message (“Hello World”) on the screen.

Input:

STDIN

```
BEGIN
  DBMS_OUTPUT.PUT_LINE('Hello World');
END;
/
```

Output:

Hello, World!

Practical No.: 04

AIM: To create a PL/SQL Program to Add Two Numbers

Input :

```
1  DECLARE
2      num1 NUMBER := 10;
3      num2 NUMBER := 20;
4      result NUMBER;
5  BEGIN
6      result := num1 + num2;
7      DBMS_OUTPUT.PUT_LINE('Sum = ' || result);
8  END;
9  /
```

Output:

Sum = 30

PL/SQL procedure successfully completed.

Practical No.: 05

AIM: To create a PL/SQL Program to Check Number is Odd or Even

Input:

```
1  DECLARE
2  |      num NUMBER := 10;
3  BEGIN
4  |      IF MOD(num, 2) = 0 THEN
5  |          DBMS_OUTPUT.PUT_LINE(num || ' is Even');
6  |      ELSE
7  |          DBMS_OUTPUT.PUT_LINE(num || ' is Odd');
8  |      END IF;
9  END;
10 /
```

Output:

10 is Even

PL/SQL procedure successfully completed.

Practical No.: 06

AIM: To find the greatest of two numbers using PL/SQL.

Input:

```
--  
3  DECLARE  
4      num1 NUMBER := 40;  
5      num2 NUMBER := 20;  
6  BEGIN  
7      IF num1 > num2 THEN  
8          DBMS_OUTPUT.PUT_LINE('Greatest number is: ' || num1);  
9      ELSIF num2 > num1 THEN  
10         DBMS_OUTPUT.PUT_LINE('Greatest number is: ' || num2);  
11     ELSE  
12         DBMS_OUTPUT.PUT_LINE('Both numbers are equal');  
13     END IF;  
14 END;  
15 /
```

Output:

Greatest number is: 40

PL/SQL procedure successfully completed.

Practical No.: 07

AIM: To implement loops in PL/SQL To find the greatest of two numbers using PL/SQL.

Input: While Loop

```
3  ✓ DECLARE
4  |      i NUMBER := 1;
5  ✓ BEGIN
6  ✓ |      WHILE i <= 5 LOOP
7  |      |      DBMS_OUTPUT.PUT_LINE('Value: ' || i);
8  |      |      i := i + 1;
9  |      END LOOP;
10 END;
11 /
```

Output:

```
Value: 1
Value: 2
Value: 3
Value: 4
Value: 5
```

PL/SQL procedure successfully completed.

For Loop:

```
3  BEGIN
4  |      FOR i IN 1..5 LOOP
5  |      |      DBMS_OUTPUT.PUT_LINE('Value: ' || i);
6  |      END LOOP;
7  END;
8  /
```

Output:

```
Value: 1  
Value: 2  
Value: 3  
Value: 4  
Value: 5
```

```
PL/SQL procedure successfully completed.
```

Practical No.: 08

AIM: To implement operators in PL/SQL

Input:

```
1  DECLARE
2      a NUMBER := 10;
3      b NUMBER := 5;
4      result NUMBER;
5  BEGIN
6      -- Arithmetic Operators
7      DBMS_OUTPUT.PUT_LINE('Addition: ' || (a + b));
8      DBMS_OUTPUT.PUT_LINE('Subtraction: ' || (a - b));
9      DBMS_OUTPUT.PUT_LINE('Multiplication: ' || (a * b));
10     DBMS_OUTPUT.PUT_LINE('Division: ' || (a / b));
11
12     -- Relational Operators
13     IF a > b THEN
14         DBMS_OUTPUT.PUT_LINE('a is greater than b');
15     END IF;
16
17     IF a = b THEN
18         DBMS_OUTPUT.PUT_LINE('a is equal to b');
19     ELSE
20         DBMS_OUTPUT.PUT_LINE('a is not equal to b');
21     END IF;
```



```

22
23      -- Logical Operators
24      IF (a > 0 AND b > 0) THEN
25      |      DBMS_OUTPUT.PUT_LINE('Both numbers are positive');
26      END IF;
27
28      IF (a > 0 OR b < 0) THEN
29      |      DBMS_OUTPUT.PUT_LINE('At least one condition is true');
30      END IF;
31
32      IF NOT (a < b) THEN
33      |      DBMS_OUTPUT.PUT_LINE('NOT condition is true');
34      END IF;
35      END;
36      /

```

Output:

```

Addition: 15
Subtraction: 5
Multiplication: 50
Division: 2
a is greater than b
a is not equal to b
Both numbers are positive
At least one condition is true
NOT condition is true

```

Practical No.: 09

AIM: To implement Switch Case Statement in PL/SQL

Input:

```
-
3  DECLARE
4      grade CHAR := 'B';
5  BEGIN
6      CASE grade
7          WHEN 'A' THEN
8              DBMS_OUTPUT.PUT_LINE('Excellent');
9          WHEN 'B' THEN
10             DBMS_OUTPUT.PUT_LINE('Very Good');
11          WHEN 'C' THEN
12             DBMS_OUTPUT.PUT_LINE('Good');
13          ELSE
14             DBMS_OUTPUT.PUT_LINE('Needs Improvement');
15      END CASE;
16  END;
17  /
```

Output:

Very Good

PL/SQL procedure successfully completed.

Elapsed: 00:00:00.008

Practical No.: 10

AIM: To implement exception handling in PL/SQL

What is Exception Handling in PL/SQL?

- An exception is an error condition during a program's execution.
- Exception handling lets you catch and handle errors gracefully without crashing the program.
- PL/SQL automatically handles internal errors, but you can also write your own custom error handlers.

Types of Exceptions:

1. Predefined Exceptions (like NO_DATA_FOUND, ZERO_DIVIDE, etc.)
2. User-Defined Exceptions (created manually by the programmer)

Example 1: Handling Division by Zero Error (ZERO_DIVIDE)

Input:

```
1  DECLARE
2      v_num1 NUMBER := 10;
3      v_num2 NUMBER := 0;
4      v_result NUMBER;
5  BEGIN
6      -- Attempt division
7      v_result := v_num1 / v_num2;
8
9      -- If no error occurs
10     DBMS_OUTPUT.PUT_LINE('Result: ' || v_result);
11
12  EXCEPTION
13     -- Handling division by zero error
14     WHEN ZERO_DIVIDE THEN
15         DBMS_OUTPUT.PUT_LINE('Error: Cannot divide by zero.');
```

Output:

Error: Cannot divide by zero.

PL/SQL procedure successfully completed.

Elapsed: 00:00:00.007

Example 2: User-Defined Exception

Input:

```
1  DECLARE
2      v_age NUMBER := 15;
3
4      -- Declare user-defined exception
5      ex_invalid_age EXCEPTION;
6  BEGIN
7      -- Check condition
8      IF v_age < 18 THEN
9          RAISE ex_invalid_age;
10     END IF;
11
12     DBMS_OUTPUT.PUT_LINE('Eligible for voting.');
```

```
14  EXCEPTION
15      -- Handle user-defined exception
16      WHEN ex_invalid_age THEN
17          DBMS_OUTPUT.PUT_LINE('Error: Age must be 18 or above.');
```

```
19      -- Optional: handle other exceptions
20      WHEN OTHERS THEN
21          DBMS_OUTPUT.PUT_LINE('An unexpected error occurred.');
```

```
22  END;
```

```
23  /
```

Output:

Error: Age must be 18 or above.

PL/SQL procedure successfully completed.

Elapsed: 00:00:00.003

Practical No.: 11

AIM: To implement various string functions in PL/SQL.

Input:

```
1  DECLARE
2      v_str1 VARCHAR2(50) := 'Hello World';
3      v_str2 VARCHAR2(50) := 'PLSQL';
4      v_result VARCHAR2(100);
5  BEGIN
6      -- UPPER
7      DBMS_OUTPUT.PUT_LINE('UPPER: ' || UPPER(v_str1));
8
9      -- LOWER
10     DBMS_OUTPUT.PUT_LINE('LOWER: ' || LOWER(v_str1));
11
12     -- LENGTH
13     DBMS_OUTPUT.PUT_LINE('LENGTH: ' || LENGTH(v_str1));
14
15     -- SUBSTR
16     DBMS_OUTPUT.PUT_LINE('SUBSTR (1,5): ' || SUBSTR(v_str1,1,5));
17
18     -- INSTR
19     DBMS_OUTPUT.PUT_LINE('INSTR (position of "World"): ' || INSTR(v_str1,'World'));
20
21     -- CONCAT
22     v_result := CONCAT(v_str1, v_str2);
23     DBMS_OUTPUT.PUT_LINE('CONCAT: ' || v_result);
24
25     -- TRIM
26     DBMS_OUTPUT.PUT_LINE('TRIM: ' || TRIM('  Oracle  '));
27
28     -- REPLACE
29     DBMS_OUTPUT.PUT_LINE('REPLACE: ' || REPLACE(v_str1,'World','PL/SQL'));
30
31 END;
32 /
```

Output:

```
UPPER: HELLO WORLD  
LOWER: hello world  
LENGTH: 11  
SUBSTR (1,5): Hello  
INSTR (position of "World"): 7  
CONCAT: Hello WorldPLSQL  
TRIM: Oracle  
REPLACE: Hello PL/SQL
```

Practical No.: 12

AIM: To implement functions and procedure in PL/SQL

What is a Function in PL/SQL?

- A Function is a named block that performs a specific task and returns a single value.
- Functions can accept input parameters, perform operations, and must return a result using the RETURN statement.
- Functions in PL/SQL always return a value.
- You can pass parameters, process data, and return results.
- Useful when you need to perform calculations or fetch single value results.

Input:

```
1  -- Create Table
2  CREATE TABLE Employees (
3      EmpID    NUMBER PRIMARY KEY,
4      EmpName  VARCHAR2(50),
5      Salary   NUMBER
6  );
7
8  -- Insert Data
9  INSERT INTO Employees VALUES (1, 'Ram', 5000);
10 INSERT INTO Employees VALUES (2, 'Abhi', 6000);
11 INSERT INTO Employees VALUES (3, 'Pooja', 7000);
12
13 COMMIT;
14
15 -- Function
16 CREATE OR REPLACE FUNCTION Get_Annual_Salary (
17     p_empid IN Employees.EmpID%TYPE
18 )
19 RETURN NUMBER
20 IS
21     v_salary Employees.Salary%TYPE;
22 BEGIN
23     SELECT Salary
24     INTO v_salary
25     FROM Employees
```



```

26         WHERE EmpID = p_empid;
27
28         RETURN v_salary * 12;
29
30     EXCEPTION
31         WHEN NO_DATA_FOUND THEN
32             DBMS_OUTPUT.PUT_LINE('Employee not found!');
33             RETURN 0;
34     END;
35 /
36
37 -- Execution
38 BEGIN
39     DBMS_OUTPUT.PUT_LINE('Annual Salary of Employee 1: ' || Get_Annual_Salary(1));
40     DBMS_OUTPUT.PUT_LINE('Annual Salary of Employee 2: ' || Get_Annual_Salary(2));
41     DBMS_OUTPUT.PUT_LINE('Annual Salary of Employee 4: ' || Get_Annual_Salary(4));
42 END;
43 /

```

Output:

```

Annual Salary of Employee 1: 60000
Annual Salary of Employee 2: 72000
Annual Salary of Employee 4: 0

```

What is a Stored Procedure in PL/SQL?

A Stored Procedure is a named block of PL/SQL code that is stored permanently in the database.

It performs a specific task, such as inserting, updating, deleting, or retrieving data. Once created, the procedure can be called (executed) whenever needed, without rewriting the code.

Key Features of Stored Procedures:

- **Reusable:** Write once, use multiple times in different programs or applications.
- **Modular:** Makes programs organized by dividing the logic into smaller parts.
- **Improves Performance:** Procedures are compiled once and stored, so execution is faster than sending multiple individual queries.

- **Security:** Database administrators can grant users permission to execute a procedure without giving direct access to the underlying tables.
- **Easy Maintenance:** If the logic changes, update the procedure at one place all programs using it will automatically pick the new logic.

Input:

```

1  -----
2  -- 1. Create Employees Table
3  -----
4  CREATE TABLE Employees (
5      EmpID    NUMBER PRIMARY KEY,
6      EmpName  VARCHAR2(50),
7      Salary   NUMBER
8  );
9  -----
10 -- 2. Insert Sample Data
11 -----
12 INSERT INTO Employees VALUES (1, 'Abhi', 5000);
13 INSERT INTO Employees VALUES (2, 'Ujjval', 6000);
14 INSERT INTO Employees VALUES (3, 'Prashant', 7000);
15
16 COMMIT;
17 -----
18 -- 3. Create Function
19 -----
20 CREATE OR REPLACE FUNCTION Get_Annual_Salary (
21     p_empid IN Employees.EmpID%TYPE
22 )
23 RETURN NUMBER

```

```

24  ✓ IS
25      v_salary Employees.Salary%TYPE;
26  ✓ BEGIN
27      SELECT Salary
28      INTO v_salary
29      FROM Employees
30      WHERE EmpID = p_empid;
31
32      RETURN v_salary * 12;
33
34  ✓ EXCEPTION
35  ✓      WHEN NO_DATA_FOUND THEN
36          DBMS_OUTPUT.PUT_LINE('Employee not found!');
37          RETURN 0;
38  END;
39  /
40  -----
41  -- 4. Execute Function
42  -----
43  ✓ BEGIN
44      DBMS_OUTPUT.PUT_LINE('Annual Salary of Employee 1: ' || Get_Annual_Salary(1));
45      DBMS_OUTPUT.PUT_LINE('Annual Salary of Employee 2: ' || Get_Annual_Salary(2));
46      DBMS_OUTPUT.PUT_LINE('Annual Salary of Employee 4: ' || Get_Annual_Salary(4));
47  END;
48  /

```

Output:

```

Annual Salary of Employee 1: 60000
Annual Salary of Employee 2: 72000
Employee not found!
Annual Salary of Employee 4: 0

```

Practical No.: 13

AIM: To implement triggers in PL/SQL

What is a Trigger in PL/SQL?

A Trigger is a special stored block of PL/SQL code that automatically executes (fires) when a specific event happens on a database table or view.

Events can be:

- INSERT (when a new record is added)
- UPDATE (when an existing record is changed)
- DELETE (when a record is removed)

You do not call a trigger manually — it activates itself automatically when the event occurs.

Key Points about Triggers:

- Automatic Execution: No need to explicitly call a trigger.
- Attached to a Table/View: Works behind the scenes whenever that table/view changes.
- Used for:
 - Data auditing (keeping track of changes)
 - Validation (checking conditions before allowing changes)
 - Enforcing business rules (like preventing changes after working hours)
- Triggers automate database operations.
- Useful for auditing, security, automatic validation.
- Fired automatically, no need to manually call them.

Input:

```

1  -- Table
2  ✓ CREATE TABLE Employees (
3      EmpID    NUMBER PRIMARY KEY,
4      EmpName  VARCHAR2(50),
5      Salary   NUMBER
6  );
7
8  -- Audit Table
9  ✓ CREATE TABLE Employee_Audit (
10     AuditID   NUMBER GENERATED ALWAYS AS IDENTITY,
11     EmpID     NUMBER,
12     EmpName   VARCHAR2(50),
13     ActionType VARCHAR2(20),
14     ActionDate DATE
15 );
16
17 -- Trigger
18 CREATE OR REPLACE TRIGGER trg_Audit_Employee
19 AFTER INSERT OR UPDATE OR DELETE
20 ON Employees
21 FOR EACH ROW
22 ✓ BEGIN
23     IF INSERTING THEN
24         INSERT INTO Employee_Audit
25             VALUES (DEFAULT, :NEW.EmpID, :NEW.EmpName, 'INSERT', SYSDATE);
26
27     ✓ ELSIF UPDATING THEN
28         INSERT INTO Employee_Audit
29             VALUES (DEFAULT, :NEW.EmpID, :NEW.EmpName, 'UPDATE', SYSDATE);
30
31     ✓ ELSIF DELETING THEN
32         INSERT INTO Employee_Audit
33             VALUES (DEFAULT, :OLD.EmpID, :OLD.EmpName, 'DELETE', SYSDATE);
34     END IF;
35 END;
36 /
37
38 -- Test
39 INSERT INTO Employees VALUES (1, 'Abhishek', 6000);
40 UPDATE Employees SET Salary = 7500 WHERE EmpID = 1;
41 DELETE FROM Employees WHERE EmpID = 1;
42
43 COMMIT;
44
45 -- View
46 SELECT * FROM Employee_Audit;

```

Output:

AUDITID	EMPID	EMPNAME	ACTIONTYPE	ACTIONDATE
1	1	Abhishek	INSERT	24-APR-26
2	1	Abhishek	UPDATE	24-APR-26
3	1	Abhishek	DELETE	24-APR-26

AUDITID	EMPID	EMPNAME	ACTIONTYPE	ACTIONDATE
1	1	Abhishek	INSERT	24-APR-26
2	1	Abhishek	UPDATE	24-APR-26
3	1	Abhishek	DELETE	24-APR-26

Practical No.: 14

AIM: To showcase a PL/SQL-based project

PL/SQL Project Theory: Student Result Management System.

Introduction: The Student Result Management System is a database application developed using PL/SQL. It is used to store, process, and manage student marks and results efficiently. The system automates tasks like result calculation, grade assignment, and data tracking.

Objectives:

To store student academic records

To calculate **total marks and percentage**

To assign grades automatically

To maintain an **audit log** of operations

To demonstrate PL/SQL concepts like procedures, functions, triggers, and exceptions.

Input:

```
1  -- Student Table
2  CREATE TABLE Students (
3      StuID    NUMBER PRIMARY KEY,
4      StuName  VARCHAR2(50),
5      Marks1   NUMBER,
6      Marks2   NUMBER,
7      Marks3   NUMBER
8  );
9
10 -- Result Table
11 CREATE TABLE Results (
12     StuID    NUMBER,
13     Total    NUMBER,
14     Percentage NUMBER,
15     Grade    VARCHAR2(2)
16 );
17
18 -- Audit Table
19 CREATE TABLE Result_Audit (
20     AuditID NUMBER GENERATED ALWAYS AS IDENTITY,
21     StuID   NUMBER,
22     Action  VARCHAR2(20),
23     Date_Time DATE
24 );
25 INSERT INTO Students VALUES (1, 'Riya', 80, 85, 90);
```

```

26  INSERT INTO Students VALUES (2, 'Aman', 60, 70, 65);
27  COMMIT;
28  CREATE OR REPLACE PROCEDURE Calculate_Result(p_id IN NUMBER)
29  IS
30      m1 NUMBER;
31      m2 NUMBER;
32      m3 NUMBER;
33      total NUMBER;
34      per NUMBER;
35      grade VARCHAR2(2);
36  BEGIN
37      SELECT Marks1, Marks2, Marks3
38      INTO m1, m2, m3
39      FROM Students
40      WHERE StuID = p_id;
41
42      total := m1 + m2 + m3;
43      per := total / 3;
44
45      -- Grade Logic
46      IF per >= 80 THEN
47          grade := 'A';
48      ELSIF per >= 60 THEN
49          grade := 'B';
50      ELSE

```

```

51          grade := 'C';
52      END IF;
53
54      INSERT INTO Results VALUES (p_id, total, per, grade);
55
56      DBMS_OUTPUT.PUT_LINE('Result Calculated for Student ' || p_id);
57
58  EXCEPTION
59      WHEN NO_DATA_FOUND THEN
60          DBMS_OUTPUT.PUT_LINE('Student not found!');
61  END;
62  /
63  CREATE OR REPLACE FUNCTION Get_Grade(p_id IN NUMBER)
64  RETURN VARCHAR2
65  IS
66      v_grade VARCHAR2(2);
67  BEGIN
68      SELECT Grade INTO v_grade
69      FROM Results
70      WHERE StuID = p_id;
71
72      RETURN v_grade;
73
74  EXCEPTION
75      WHEN NO_DATA_FOUND THEN

```



```

76 | | RETURN 'NA';
77 | END;
78 | /
79 | CREATE OR REPLACE TRIGGER trg_Result_Audit
80 | AFTER INSERT OR UPDATE OR DELETE
81 | ON Results
82 | FOR EACH ROW
83 | BEGIN
84 |     IF INSERTING THEN
85 |         INSERT INTO Result_Audit VALUES (DEFAULT, :NEW.StuID, 'INSERT', SYSDATE);
86 |     ELSIF UPDATING THEN
87 |         INSERT INTO Result_Audit VALUES (DEFAULT, :NEW.StuID, 'UPDATE', SYSDATE);
88 |     ELSIF DELETING THEN
89 |         INSERT INTO Result_Audit VALUES (DEFAULT, :OLD.StuID, 'DELETE', SYSDATE);
90 |     END IF;
91 | END;
92 | /
93 | BEGIN
94 |     Calculate_Result(1);
95 |     Calculate_Result(2);
96 |
97 |     DBMS_OUTPUT.PUT_LINE('Grade of Student 1: ' || Get_Grade(1));
98 | END;
99 | /

101 | -- View Results
102 | SELECT * FROM Results;
103 |
104 | -- View Audit
105 | SELECT * FROM Result_Audit;

```

Output:

```

Result Calculated for Student 1
Result Calculated for Student 2
Grade of Student 1: A

```

STUID	TOTAL	PERCENTAGE	GR
1	255	85	A
2	195	65	B